

Python pathlib practice

```
"""
=====
PYTHON pathlib - TASKS + SOLUTIONS
=====

Goal:
    Practice pathlib through practical Data Engineering tasks.

Important:
    pathlib = working with files, folders, and paths.

Run this entire script.

=====
"""

from pathlib import Path
import json

# =====
# SETUP
# =====

BASE_DIR = Path("pathlib_practice")

RAW_DIR = BASE_DIR / "raw"
PROCESSED_DIR = BASE_DIR / "processed"
ARCHIVE_DIR = BASE_DIR / "archive"

RAW_DIR.mkdir(parents=True, exist_ok=True)
PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
ARCHIVE_DIR.mkdir(parents=True, exist_ok=True)

# =====
# TASK 1
# =====

"""
TASK 1
-----
Create a Path object representing:

    data/raw/events.json
"""

# SOLUTION

file_path = Path("data") / "raw" / "events.json"

print("\nTASK 1")
print(file_path)
```

```

# =====
# TASK 2
# =====

"""
TASK 2
-----
Create these directories:

    pathlib_practice/
    raw/
    processed/
    archive/
"""

# SOLUTION

base = Path("pathlib_practice")

raw = base / "raw"
processed = base / "processed"
archive = base / "archive"

raw.mkdir(parents=True, exist_ok=True)
processed.mkdir(parents=True, exist_ok=True)
archive.mkdir(parents=True, exist_ok=True)

print("\nTASK 2")
print("Directories created.")

# =====
# TASK 3
# =====

"""
TASK 3
-----
Create:

    raw/events.json
"""

# SOLUTION

events_file = RAW_DIR / "events.json"

events_file.touch()

print("\nTASK 3")
print(events_file)

```

```

# =====
# TASK 4
# =====

"""
TASK 4
-----
Check whether events.json exists.
"""

# SOLUTION

print("\nTASK 4")

if events_file.exists():
    print("events.json exists")
else:
    print("events.json does not exist")

# =====
# TASK 5
# =====

"""
TASK 5
-----
Check whether events.json is a file.
"""

# SOLUTION

print("\nTASK 5")
print(events_file.is_file())

# =====
# TASK 6
# =====

"""
TASK 6
-----
Check whether raw is a directory.
"""

# SOLUTION

print("\nTASK 6")
print(RAW_DIR.is_dir())

# =====
# TASK 7
# =====

"""
TASK 7
-----
Print:

    filename
    extension
    filename without extension
    parent directory
"""

# SOLUTION

```

```

print("\nTASK 7")

print("Name:", events_file.name)
print("Suffix:", events_file.suffix)
print("Stem:", events_file.stem)
print("Parent:", events_file.parent)

# =====
# TASK 8
# =====

"""
TASK 8
-----
Write this text into events.json:

    Hello Data Engineering
"""

# SOLUTION

events_file.write_text("Hello Data Engineering")

print("\nTASK 8")
print(events_file.read_text())

# =====
# TASK 9
# =====

"""
TASK 9
-----
Create a JSON file containing:

[
    {"id": 1, "value": 100},
    {"id": 2, "value": 200},
    {"id": 3, "value": 300}
]
"""

# SOLUTION

events = [
    {"id": 1, "value": 100},
    {"id": 2, "value": 200},
    {"id": 3, "value": 300}
]

with events_file.open("w") as file:
    json.dump(events, file, indent=4)

print("\nTASK 9")
print(events_file.read_text())

# =====
# TASK 10
# =====

"""
TASK 10
-----
Read the JSON file and print the data.
"""

```

```

# SOLUTION

with events_file.open("r") as file:
    data = json.load(file)

print("\nTASK 10")

for event in data:
    print(event)

# =====
# TASK 11
# =====

"""
TASK 11
-----
Create these files:

    raw/customers.json
    raw/products.json
    raw/orders.json
"""

# SOLUTION

print("\nTASK 11")

customers_file = RAW_DIR / "customers.json"
products_file = RAW_DIR / "products.json"
orders_file = RAW_DIR / "orders.json"

customers_file.touch()
products_file.touch()
orders_file.touch()

print("Files created.")

# =====
# TASK 12
# =====

"""
TASK 12
-----
List everything inside raw/.
"""

# SOLUTION

print("\nTASK 12")

for item in RAW_DIR.iterdir():
    print(item)

# =====
# TASK 13
# =====

"""
TASK 13
-----
Print only files inside raw/.
"""

```

```

# SOLUTION

print("\nTASK 13")

for item in RAW_DIR.iterdir():
    if item.is_file():
        print(item)

# =====
# TASK 14
# =====

"""
TASK 14
-----
Find only JSON files inside raw/.
"""

# SOLUTION

print("\nTASK 14")

for file in RAW_DIR.glob("*.json"):
    print(file)

# =====
# TASK 15
# =====

"""
TASK 15
-----
Create these files:

    raw/events.csv
    raw/customers.csv
    raw/products.csv

Then find only CSV files.
"""

# SOLUTION

csv_files = [
    RAW_DIR / "events.csv",
    RAW_DIR / "customers.csv",
    RAW_DIR / "products.csv"
]

for file in csv_files:
    file.touch()

print("\nTASK 15")

for file in RAW_DIR.glob("*.csv"):
    print(file)

```

```

# =====
# TASK 16
# =====

"""
TASK 16
-----
Create nested directories:

    raw/2026/09/07/

Then create:

    events.json
"""

# SOLUTION

print("\nTASK 16")

date_dir = RAW_DIR / "2026" / "09" / "07"
date_dir.mkdir(parents=True, exist_ok=True)
daily_file = date_dir / "events.json"
daily_file.touch()
print(daily_file)

# =====
# TASK 17
# =====

"""
TASK 17
-----
Find ALL JSON files recursively inside raw/.

Hint:
    rglob()
"""

# SOLUTION

print("\nTASK 17")

for file in RAW_DIR.rglob("*.json"):
    print(file)

# =====
# TASK 18
# =====

"""
TASK 18
-----
Rename:

    customers.json

to:

    customers_processed.json
"""

```

```

# SOLUTION

print("\nTASK 18")

old_file = RAW_DIR / "customers.json"
new_file = RAW_DIR / "customers_processed.json"

old_file.rename(new_file)

print("Renamed:")
print(new_file)

# =====
# TASK 19
# =====

"""
TASK 19
-----
Move:

    products.json

from raw/

to processed/.
"""

# SOLUTION

print("\nTASK 19")

source = RAW_DIR / "products.json"
destination = PROCESSED_DIR / "products.json"

source.rename(destination)

print("Moved to:")
print(destination)

# =====
# TASK 20
# =====

"""
TASK 20
-----
Change:

    events.csv

to:

    events.json

without changing the directory.
"""

# SOLUTION

print("\nTASK 20")

csv_file = RAW_DIR / "events.csv"

json_version = csv_file.with_suffix(".json")

print("Original:", csv_file)
print("New:", json_version)

```



```

# =====
# TASK 21
# =====

"""
TASK 21
-----
Change:

    events.json

to:

    events_processed.json
"""

# SOLUTION

print("\nTASK 21")

file = RAW_DIR / "events.json"
renamed = file.with_name("events_processed.json")

print("Original:", file)
print("New:", renamed)

# =====
# TASK 22
# =====

"""
TASK 22
-----
Print every part of this path:

    raw/2026/09/07/events.json
"""

# SOLUTION

print("\nTASK 22")

file = RAW_DIR / "2026" / "09" / "07" / "events.json"

for part in file.parts:
    print(part)

# =====
# TASK 23
# =====

"""
TASK 23
-----
Print:

    parent
    grandparent
    filename
"""

# SOLUTION

print("\nTASK 23")

print("Parent:", file.parent)

```

```

print("Grandparent:", file.parent.parent)
print("Filename:", file.name)

# =====
# TASK 24
# =====

"""
TASK 24
-----
Get the absolute path of:

    raw/events.json
"""

# SOLUTION

print("\nTASK 24")

file = RAW_DIR / "events.json"
print(file.resolve())

# =====
# TASK 25
# =====

"""
TASK 25
-----
Create 5 JSON files:

    event_1.json
    event_2.json
    event_3.json
    event_4.json
    event_5.json
"""

# SOLUTION

print("\nTASK 25")

for i in range(1, 6):
    file = RAW_DIR / f"event_{i}.json"

    file.write_text(
        json.dumps(
            {
                "id": i,
                "value": i * 100
            },
            indent=4
        )
    )

    print("Created:", file)

# =====
# TASK 26
# =====

```

```

"""
TASK 26
-----
Loop through all event_*.json files
and print their contents.
"""

# SOLUTION

print("\nTASK 26")

for file in RAW_DIR.glob("event_*.json"):
    print("\nProcessing:", file)

    data = json.loads(file.read_text())

    print(data)

# =====
# TASK 27
# =====

"""
TASK 27
-----
Copy the idea of a simple Data Engineering pipeline:

    raw
    ↓
    processed
    ↓
    archive

For every event_*.json:

1. Read the JSON.
2. Write it into processed/.
3. Move the original into archive/.
"""

# SOLUTION

print("\nTASK 27")

for file in RAW_DIR.glob("event_*.json"):
    print("\nProcessing:", file.name)

    # -----
    # 1. Read
    # -----

    data = json.loads(file.read_text())

    # -----
    # 2. Write to processed
    # -----

    processed_file = PROCESSED_DIR / file.name

    processed_file.write_text(
        json.dumps(data, indent=4)
    )

    print("Processed:", processed_file)

```

```

# -----
# 3. Move original
# -----

archive_file = ARCHIVE_DIR / file.name

file.rename(archive_file)

print("Archived:", archive_file)

# =====
# TASK 28
# =====

"""
TASK 28
-----
Find all JSON files recursively inside:

    pathlib_practice/

Print their paths.
"""

# SOLUTION

print("\nTASK 28")

for file in BASE_DIR.rglob("*.json"):
    print(file)

# =====
# TASK 29
# =====

"""
TASK 29
-----
Print the size of every JSON file in bytes.
"""

# SOLUTION

print("\nTASK 29")

for file in BASE_DIR.rglob("*.json"):
    size = file.stat().st_size
    print(file, "->", size, "bytes")

# =====
# TASK 30
# =====

"""
TASK 30
-----
Create a simple file-processing function:

    process_json_files(directory)

The function should:

    - receive a directory
    - find JSON files
    - print each filename
"""

```

```

# SOLUTION

def process_json_files(directory):
    directory = Path(directory)
    for file in directory.glob("*.json"):
        print("Processing:", file.name)

print("\nTASK 30")
process_json_files(RAW_DIR)

# =====
# TASK 31
# =====

"""
TASK 31
-----
Create a function:

    get_json_files(directory)

It should return a list containing
all JSON files.
"""

# SOLUTION

def get_json_files(directory):
    directory = Path(directory)
    return list(directory.glob("*.json"))

print("\nTASK 31")
json_files = get_json_files(BASE_DIR)
for file in json_files:
    print(file)

# =====
# TASK 32 - REAL DATA ENGINEERING SCENARIO
# =====

"""
TASK 32
-----
Build this directory structure:

    data/
    raw/
    silver/
    gold/
    archive/

This represents a simple data pipeline.

raw:
    incoming data

silver:

```

```

        cleaned/transformed data

gold:
    business-ready data

archive:
    old/raw processed files
"""

# SOLUTION

print("\nTASK 32")

data = Path("data")

layers = [
    "raw",
    "silver",
    "gold",
    "archive"
]

for layer in layers:
    directory = data / layer
    directory.mkdir(parents=True, exist_ok=True)
    print("Created:", directory)

# =====
# TASK 33 – BUILD A DYNAMIC PATH
# =====

"""
TASK 33
-----
Build this path dynamically:

    data/raw/2026/09/events.json

Don't manually type the complete path.
"""

# SOLUTION

print("\nTASK 33")

year = "2026"
month = "09"

path = (
    data
    / "raw"
    / year
    / month
    / "events.json"
)

print(path)

```

```

# =====
# TASK 34 - FILE EXTENSION CHECK
# =====

"""
TASK 34
-----
Find all files in raw/ and print:

    filename
    extension

Example:

    events.json -> .json
"""

# SOLUTION

print("\nTASK 34")

for file in RAW_DIR.rglob("*"):
    if file.is_file():
        print(
            file.name,
            "->",
            file.suffix
        )

# =====
# TASK 35 - DELETE A FILE
# =====

"""
TASK 35
-----
Create:

    temporary.txt

Then delete it.
"""

# SOLUTION

print("\nTASK 35")

temporary_file = BASE_DIR / "temporary.txt"
temporary_file.write_text("temporary data")
print("Exists:", temporary_file.exists())
temporary_file.unlink()
print("Exists after deletion:", temporary_file.exists())

# =====
# FINAL DATA ENGINEERING EXERCISE
# =====

"""
=====
FINAL PROJECT
=====

```

Build this structure:

```
pipeline/  
  raw/  
    processed/  
    archive/
```

Create 3 JSON files in raw/:

```
sales_1.json  
sales_2.json  
sales_3.json
```

Each file should contain:

```
{  
  "id": number,  
  "amount": number  
}
```

Then:

1. Find all JSON files in raw/.

2. Read each JSON file.

3. Add:

```
  "status": "processed"
```

4. Save the modified JSON into processed/.

5. Move the original file into archive/.

Expected flow:

```
raw/  
  sales_1.json  
  sales_2.json  
  sales_3.json
```

↓

```
processed/  
  sales_1.json  
  sales_2.json  
  sales_3.json
```

+

```
archive/  
  sales_1.json  
  sales_2.json  
  sales_3.json
```

=====

SOLUTION

=====

"""

```
pipeline = Path("pipeline")
```

```
pipeline_raw = pipeline / "raw"
```

```
pipeline_processed = pipeline / "processed"
```

```
pipeline_archive = pipeline / "archive"
```

```
pipeline_raw.mkdir(parents=True, exist_ok=True)
```

```
pipeline_processed.mkdir(parents=True, exist_ok=True)
```



```
pipeline_archive.mkdir(parents=True, exist_ok=True)
```

```
# Create sample sales files
```

```
for i in range(1, 4):
```

```
    file = pipeline_raw / f"sales_{i}.json"
```

```
    data = {  
        "id": i,  
        "amount": i * 100  
    }
```

```
    file.write_text(  
        json.dumps(data, indent=4)  
    )
```

```
# Process files
```

```
for file in pipeline_raw.glob("*.json"):
```

```
    print("\nProcessing:", file.name)
```

```
    # -----  
    # READ  
    # -----
```

```
    data = json.loads(  
        file.read_text()  
    )
```

```
    # -----  
    # TRANSFORM  
    # -----
```

```
    data["status"] = "processed"
```

```
    # -----  
    # WRITE  
    # -----
```

```
    processed_file = (  
        pipeline_processed / file.name  
    )
```

```
    processed_file.write_text(  
        json.dumps(data, indent=4)  
    )
```

```
    # -----  
    # ARCHIVE  
    # -----
```

```
    archive_file = (  
        pipeline_archive / file.name  
    )
```

```
    file.rename(archive_file)
```

```
    print("Processed:", processed_file)  
    print("Archived:", archive_file)
```

```

# =====
# FINAL SUMMARY
# =====

print("\n")
print("=" * 60)
print("PATHLIB PRACTICE COMPLETED")
print("=" * 60)

print(
    """
Most important pathlib skills:

Path()
    Create paths.

/
    Combine paths.

mkdir()
    Create directories.

exists()
    Check whether something exists.

is_file()
    Check whether path is a file.

is_dir()
    Check whether path is a directory.

iterdir()
    List directory contents.

glob()
    Find files using patterns.

rglob()
    Find files recursively.

read_text()
    Read text.

write_text()
    Write text.

open()
    Open files.

rename()
    Rename or move files.

unlink()
    Delete files.

name
    Filename.

stem
    Filename without extension.

suffix
    File extension.

suffixes
    Multiple extensions.

parent
    Parent directory.

```

parts
Individual path components.

resolve()
Get absolute path.

stat()
Get file metadata.

with_name()
Change filename.

with_suffix()
Change extension.

DATA ENGINEERING PATTERN

Pathlib is especially important for:

```
raw files
  ↓
cleaning
  ↓
processed files
  ↓
archive
```

Example:

```
raw_dir = Path("data/raw")

for file in raw_dir.glob("*.json"):
    data = json.loads(file.read_text())

    # transform data

    output = Path("data/processed") / file.name

    output.write_text(
        json.dumps(data)
    )
```

NEXT

After pathlib, a good Python Data Engineering sequence is:

```
pathlib
  ↓
os
  ↓
time
  ↓
datetime
  ↓
json
  ↓
csv
```

You already worked through datetime and time,
so pathlib + json + datetime will be particularly

```
useful for your upcoming file-generation pipeline.  
""  
)
```